Natural Language Processing

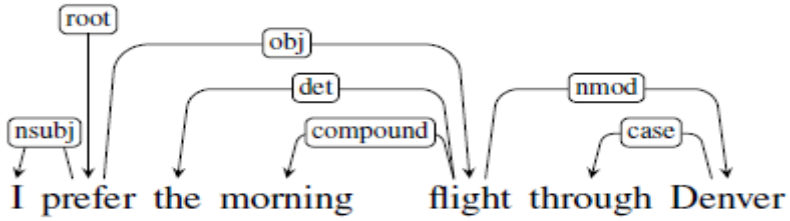
Prof. Ahmed Guessoum
The National Higher School of AI

Chapter 19

Dependency Parsing

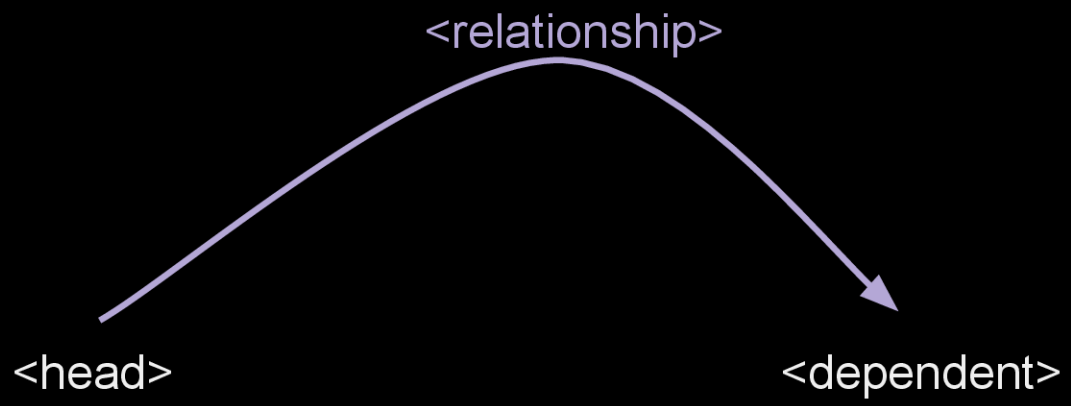
Dependency Grammars

- Dependency grammars represent an important family of grammar formalisms.
- Syntactic structure of a sentence is described solely in terms of directed binary grammatical relations between the words.



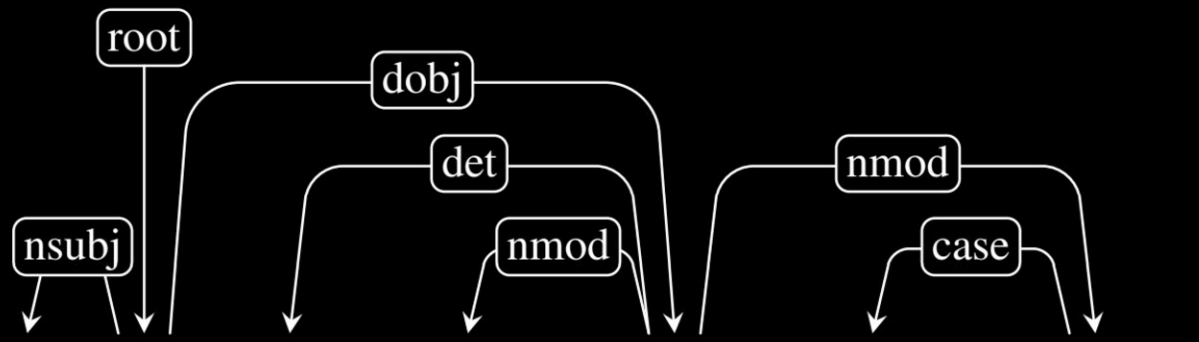
- **Relations** among the words illustrated with directed, labeled typed arcs from **heads** to **dependents**. (No phrasal constituents here.)
- *root* node explicitly marks the root of the tree, i.e. the head of the entire structure.
- Dependency grammars currently more common than constituency grammars in NLP because:
  - Relations are a good proxy for semantic relationship between words.
  - They can deal with languages that have a relatively free word order.

Dependency Parsing



dependency -- binary asymmetrical relation between tokens

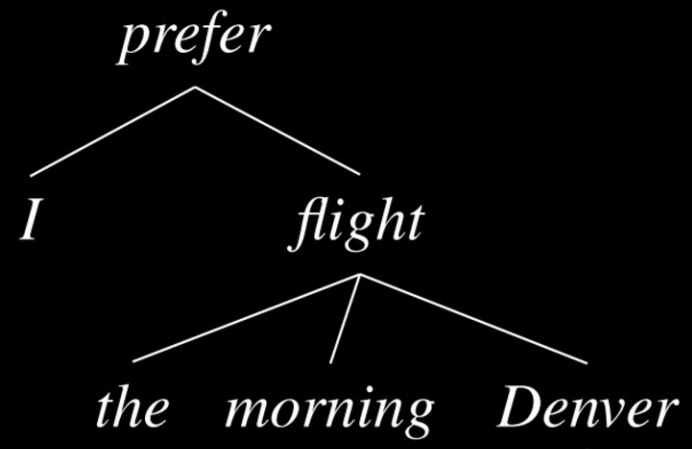
Dependency Parsing

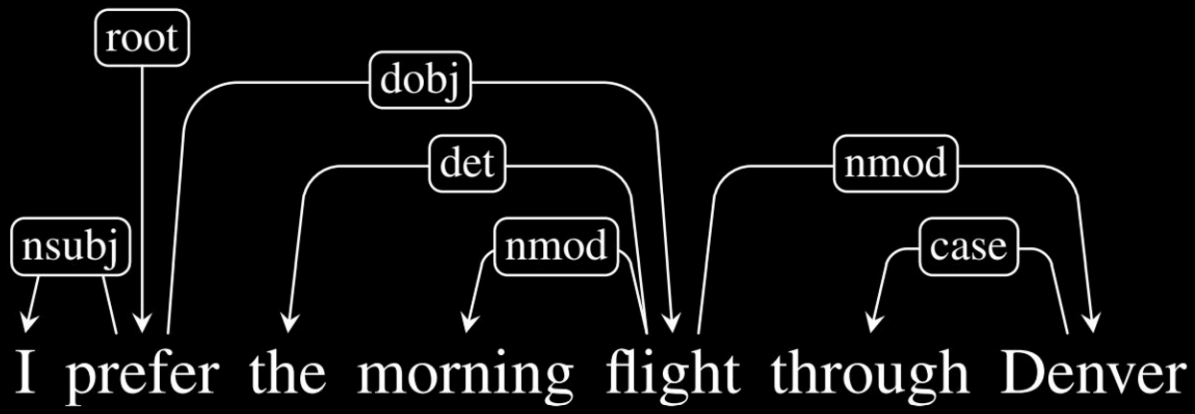


(13.1)

(From SLP 3rd ed., Jurafsky and Martin 2018)

Dependency Parsing





(13.1)

(From SLP 3rd ed., Jurafsky and Martin 2018)

Dependency Relations

- The *head* and *dependent* are the arguments of the traditional notion of *grammatical relation*.
- **Head** plays the role of the central organizing word; the **dependent** a kind of modifier.
- Head-dependent relationship made explicit by directly linking heads to the words that are immediately dependent on them.
- Dependency grammars indicate the grammatical function that a dependent plays with respect to its head (e.g. subject, direct object indirect object, …).
- Linguists have developed taxonomies of relations that go well beyond the familiar notions of subject and object.
- The Universal Dependencies project is an open community effort to annotate dependencies (+ other aspects) across 100+ languages, using 37 dependency relations.

Some Universal Dependency relations

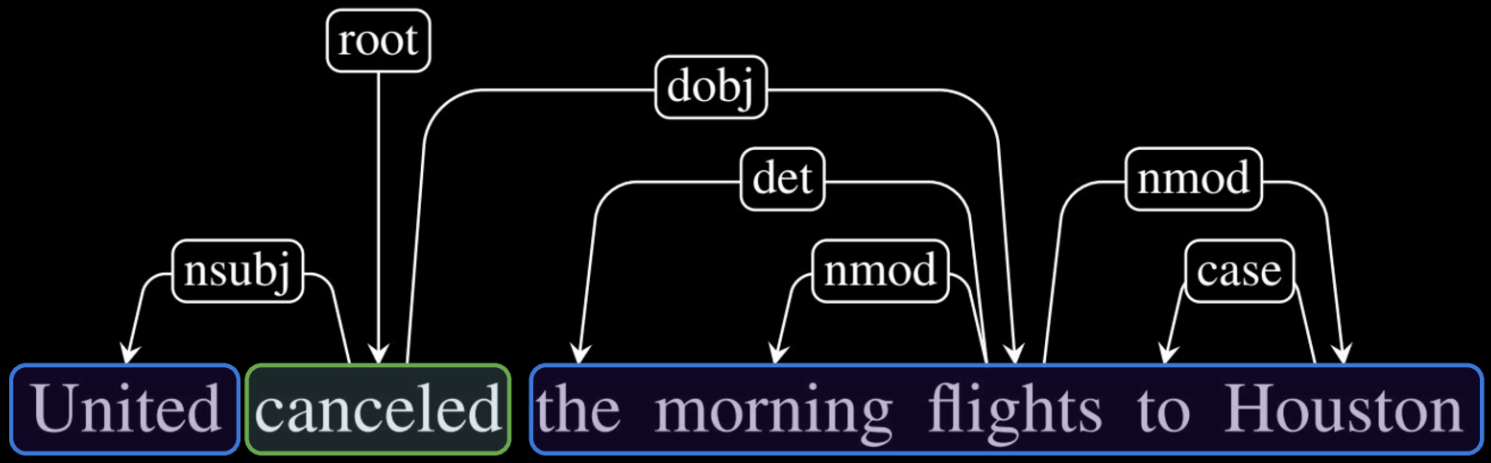
Clausal Argument Relations	Description
NSUBJ	Nominal subject
OBJ	Direct object
IOBJ	Indirect object
CCOMP	Clausal complement
Nominal Modifier Relations	**Description**
NMOD	Nominal modifier
AMOD	Adjectival modifier
APPOS	Appositional modifier
DET	Determiner
CASE	Prepositions, postpositions and other case markers
Other Notable Relations	**Description**
CONJ	Conjunct
CC	Coordinating conjunction

Examples of Universal Dependency relations

Relation	Examples with *head* and **dependent**
NSUBJ	**United** *canceled* the flight.
OBJ	United *diverted* the **flight** to Reno.
	We *booked* her the first **flight** to Miami.
IOBJ	We *booked* **her** the flight to Miami.
COMPOUND	We took the **morning** *flight*.
NMOD	*flight* to **Houston**.
AMOD	Book the **cheapest** *flight*.
APPOS	*United*, a **unit** of UAL, matched the fares.
DET	**The** *flight* was canceled.
	Which *flight* was delayed?
CONJ	We *flew* to Denver and **drove** to Steamboat.
CC	We flew to Denver **and** *drove* to Steamboat.
CASE	Book the flight **through** *Houston*.

Dependency Parsing -- Verbal Predicates

cancel(“United”, “the morning flights to Houston”)

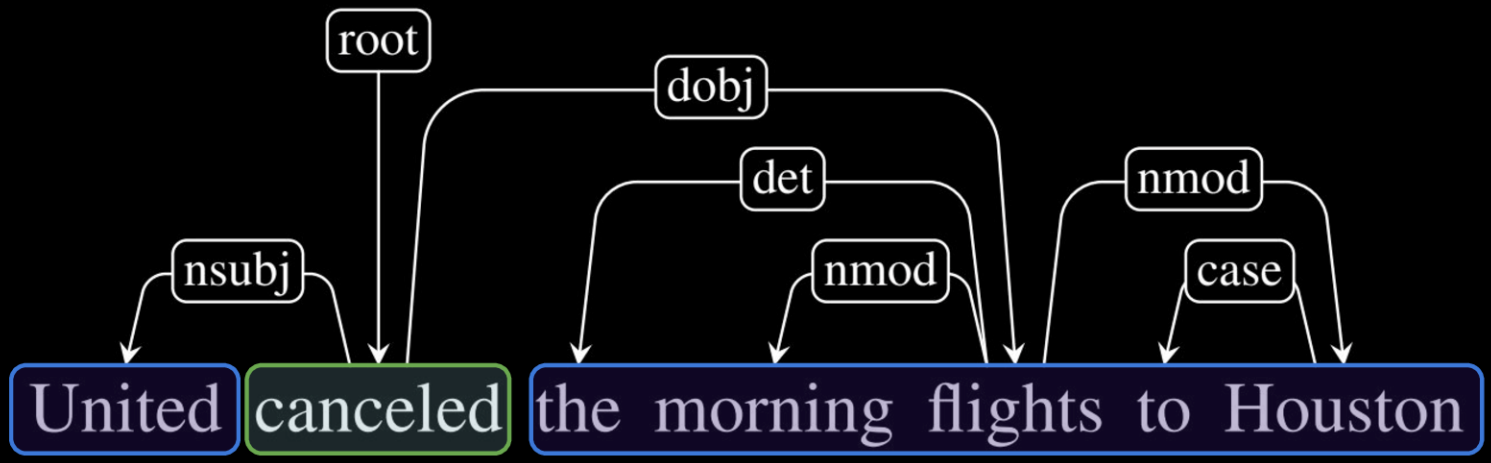


(13.2)

(From SLP 3rd ed., Jurafsky and Martin 2018)

Dependency Parsing -- Verbal Predicates

to_call_off("United", "the morning flights to Houston")

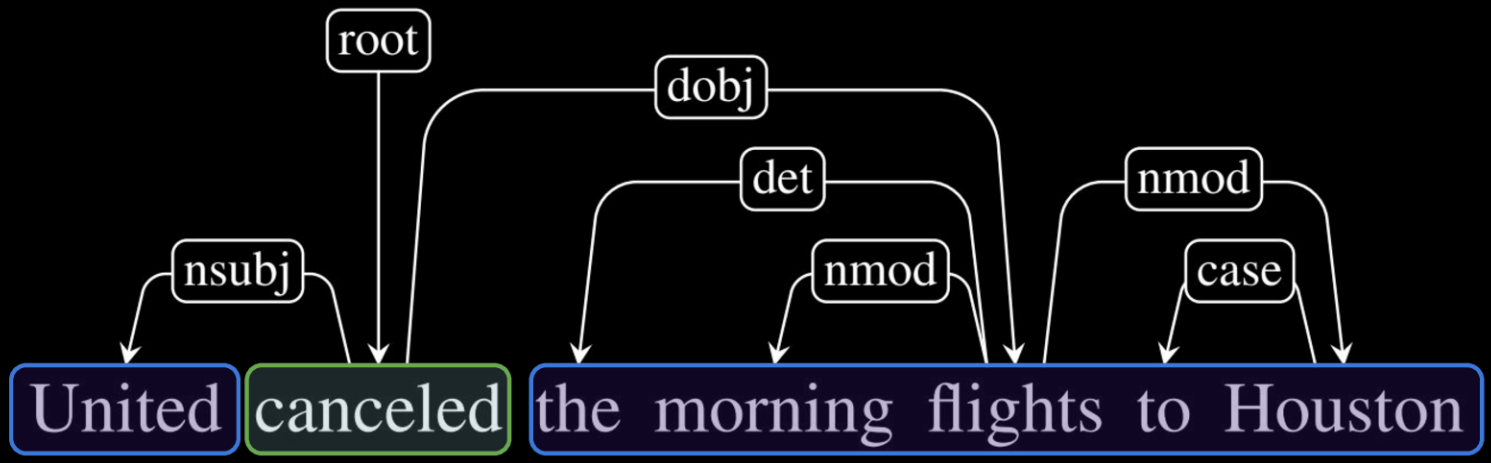


(13.2)

(From SLP 3rd ed., Jurafsky and Martin 2018)

Dependency Parsing -- Verbal Predicates
Semantic Roles

to_call_off(agent="United", event="the morning flights to Houston")



(13.2)

(From SLP 3rd ed., Jurafsky and Martin 2018)

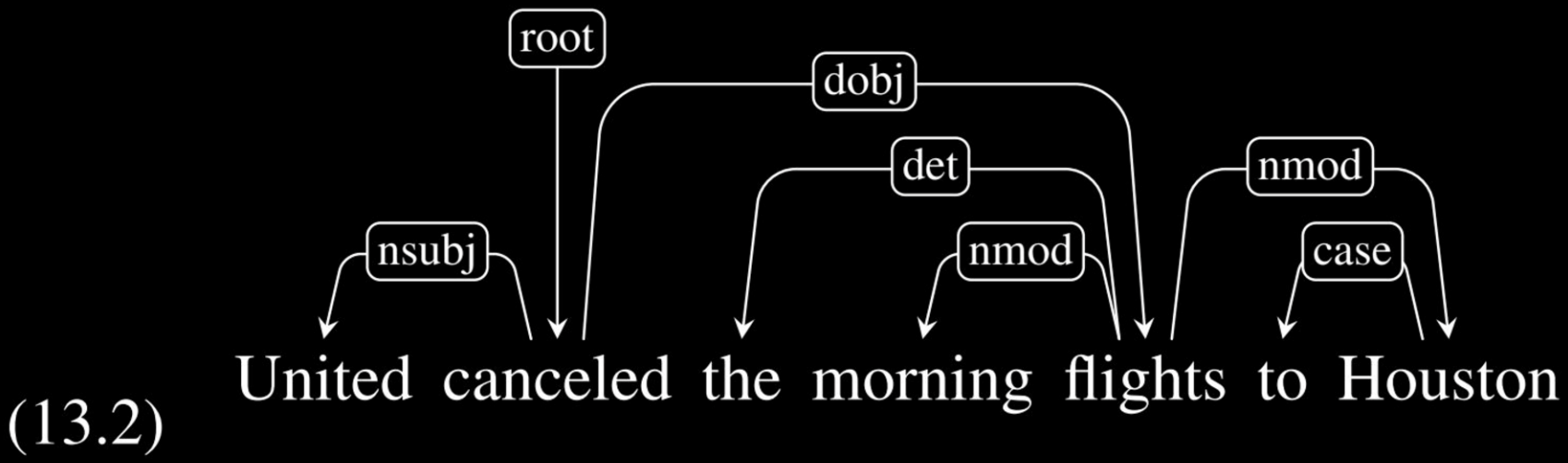
Dependency Formalisms

- A dependency structure can be represented as a directed graph G=(V, A)
  - V: a set of vertices, and
  - A: a set of ordered pairs of vertices, called arcs.
- We will assume that V corresponds to
  - the set of words in a given sentence,
  - punctuation, or
  - (for morphologically complex languages) stems and affixes.
- A captures the head-dependent and grammatical function relationships between the elements in V.

Dependency Parsing -- How to Represent?

A Graph: G = [(V1, A1), (V2, A2), …] (vertices and arcs)
Restrictions:

1) Single designated ROOT with no incoming arcs
2) Every vertex only has one head (parent, governer); i.e. only one incoming arc
3) unique path from ROOT to every vertex



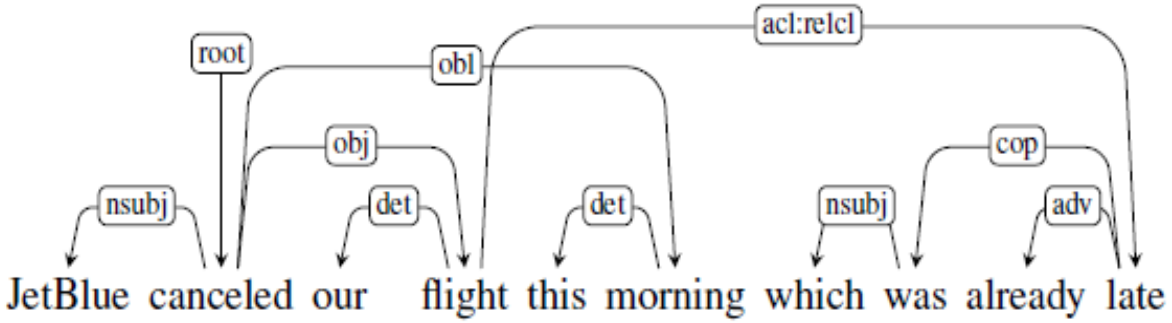
(13.2)

(From SLP 3rd ed., Jurafsky and Martin 2018)

(2) … except for the root.

Projectivity

- ***Definition***: *An arc from a head to a dependent is said to be projective if there is a path from the head to every word that lies between the head and the dependent in the sentence.*
- A dependency tree is said to be projective if all the arcs that make it up are projective.



- The arc from *flight* to its modifier *late* is non-projective since there is no path from *flight* to the intervening words *this* and *morning*.
- Detection of projectivity: A dependency tree is projective if it can be drawn with no crossing edges.
- There are many valid constructions which lead to non-projective trees, particularly in languages with relatively flexible word order.
- The transition-based approaches discussed below can only produce projective trees

Dependency Treebanks

- Dependency treebanks are created by
  - human annotators who directly generate dependency structures for a given corpus, or
  - by hand-correcting the output of an automatic parser.
  - using a deterministic process to translate existing constituent-based treebanks into dependency trees. (This was the case for a few early treebanks.)
- The largest open community project for building dependency trees is the Universal Dependencies project (https://universaldependencies.org/): it currently has almost 200 dependency treebanks in more than 100 languages

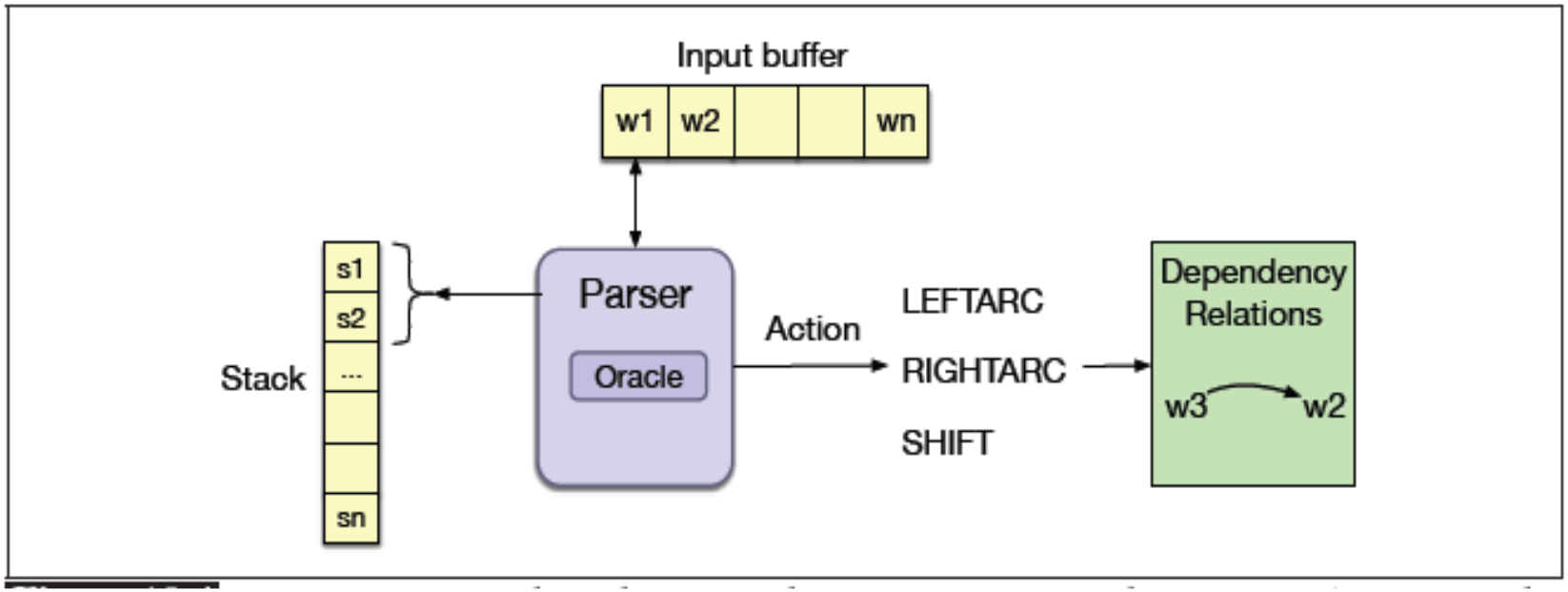
Transition-based Dependency Parsing

Inspired by “Shift-reduce parsing” -- process one word at a time, using a stack to keep some sort of memory.

Elements:

- *S*: stack, initialized with “ROOT”
- *B*: input buffer, initialized with tokens (w1, w2, ….) of sentence
- *A:* set of dependency arcs, initialized empty
- *T:* Actions, given *wi* (next token in stack)

Basic Transition-Based Parser



- The parser walks through the sentence left-to-right, successively shifting items from the buffer onto the stack.
- At each time point, we examine the top two elements on the stack, and
- The oracle makes a decision about what transition to apply to build the parse.

Basic Transition-Based Parser

- The parser uses three transition operators that act on the top two elements of the stack:
  - **LEFTARC**: Assert a head-dependent relation between the word at the top of the stack and the second word; remove the second word from the stack;
  - **RIGHTARC**: Assert a head-dependent relation between the second word on the stack and the word at the top; remove the top word from the stack;
  - **SHIFT**: Remove the word from the front of the input buffer and push it onto the stack.
- Preconditions for using the operators:
  - The LEFTARC operator cannot be applied when ROOT is the second element of the stack (since by definition the ROOT node cannot have any incoming arcs).
  - Both the LEFTARC and RIGHTARC operators require two elements to be on the stack to be applied.

A Generic Transition-based Dependency Parser

- The specification of a transition-based parser is based on representing the current state of the parse as a **configuration**: the stack, an input buffer of words or tokens, and a set of relations representing a dependency tree.
- Parsing: making a sequence of transitions through the space of possible configurations.
- In the final goal state, the stack and the word list should be empty, and the set of relations will represent the final parse.

```
function DEPENDENCYPARSE(words) returns dependency tree

  state ← {[root], [words], [] }  ; initial configuration
  while state not final
    t ← ORACLE(state)         ; choose a transition operator to apply
    state ← APPLY(t, state)   ; apply it, creating a new state
  return state
```

Transition-based Dependency Parser

- At each step, the parser
  - consults an oracle (coming next) that provides the correct transition operator to use given the current configuration;
  - then applies that operator to the current configuration, producing a new configuration.
- The process ends when all the words in the sentence have been consumed and the ROOT node is the only element remaining on the stack.
- This approach is a straightforward greedy algorithm—the oracle provides a single choice at each step and the parser proceeds with that choice, no other options are explored, no backtracking is employed, and a single parse is returned in the end.

Transition-based Dependency Parsing

Step	Stack	Word List	Action	Relation Added
0	[root]	[book, me, the, morning, flight]	SHIFT	

shift(*B,S*): move w from *B* to *S*

left-arc(S,A): make top of stack **head** of next item: add to A;
remove dependent from stack

right-arc(S,A): make top of stack **dependent** of next item: add to A;
remove dep from stack

(From SLP 3rd ed., Jurafsky and Martin 2018)

Transition-based Dependency Parsing

Step	Stack	Word List	Action	Relation Added
0	[root]	[book, me, the, morning, flight]	SHIFT	
1	[root, book]	[me, the, morning, flight]	SHIFT	

shift(*B,S*): move w from *B* to *S*

left-arc(S,A): make top of stack **head** of next item: add to A;
remove dependent from stack

right-arc(S,A): make top of stack **dependent** of next item: add to A;
remove dep from stack

(From SLP 3rd ed., Jurafsky and Martin 2018)

Transition-based Dependency Parsing

Step	Stack	Word List	Action	Relation Added
0	[root]	[book, me, the, morning, flight]	SHIFT	
1	[root, book]	[me, the, morning, flight]	SHIFT	
2	[root, book, me]	[the, morning, flight]	RIGHTARC	(book → me)

shift(*B*,*S*): move w from *B* to *S*

left-arc(S,A): make top of stack **head** of next item: add to A;
remove dependent from stack

right-arc(S,A): make top of stack **dependent** of next item: add to A;
remove dep from stack

(From SLP 3rd ed., Jurafsky and Martin 2018)

Transition-based Dependency Parsing

Step	Stack	Word List	Action	Relation Added
0	[root]	[book, me, the, morning, flight]	SHIFT	
1	[root, book]	[me, the, morning, flight]	SHIFT	
2	[root, book, me]	[the, morning, flight]	RIGHTARC	(book → me)
3	[root, book]	[the, morning, flight]	SHIFT	
4	[root, book, the]	[morning, flight]	SHIFT	
5	[root, book, the, morning]	[flight]	SHIFT	

shift(*B,S*): move w from *B* to *S*

left-arc(S,A): make top of stack **head** of next item: add to A;
remove dependent from stack

right-arc(S,A): make top of stack **dependent** of next item: add to A;
remove dep from stack

(From SLP 3rd ed., Jurafsky and Martin 2018)

Transition-based Dependency Parsing

Step	Stack	Word List	Action	Relation Added
0	[root]	[book, me, the, morning, flight]	SHIFT	
1	[root, book]	[me, the, morning, flight]	SHIFT	
2	[root, book, me]	[the, morning, flight]	RIGHTARC	(book → me)
3	[root, book]	[the, morning, flight]	SHIFT	
4	[root, book, the]	[morning, flight]	SHIFT	
5	[root, book, the, morning]	[flight]	SHIFT	
6	[root, book, the, morning, flight]	[]	LEFTARC	(morning ← flight)

shift(*B,S*): move w from *B* to *S*

left-arc(S,A): make top of stack **head** of next item: add to A;
remove dependent from stack

right-arc(S,A): make top of stack **dependent** of next item: add to A;
remove dep from stack

(From SLP 3rd ed., Jurafsky and Martin 2018)

Transition-based Dependency Parsing

Step	Stack	Word List	Action	Relation Added
0	[root]	[book, me, the, morning, flight]	SHIFT	
1	[root, book]	[me, the, morning, flight]	SHIFT	
2	[root, book, me]	[the, morning, flight]	RIGHTARC	(book → me)
3	[root, book]	[the, morning, flight]	SHIFT	
4	[root, book, the]	[morning, flight]	SHIFT	
5	[root, book, the, morning]	[flight]	SHIFT	
6	[root, book, the, morning, flight]	[]	LEFTARC	(morning ← flight)
7	[root, book, the, flight]	[]	LEFTARC	(the ← flight)
8	[root, book, flight]	[]	RIGHTARC	(book → flight)
9	[root, book]	[]	RIGHTARC	(root → book)
10	[root]	[]	Done	

Figure 13.7 Trace of a transition-based parse.

(From SLP 3rd ed., Jurafsky and Martin 2018)

Creating an Oracle

- The oracle for greedily selecting the appropriate transition is trained by supervised ML.
- So we need training data: configurations annotated with the correct transitions to take.
- We can draw these from dependency trees.
- Unfortunately, treebanks pair entire sentences with their corresponding trees, not configurations with transitions.
- To generate the required training data, supply the oracle with the training sentences to be parsed along with their corresponding reference parses from the treebank.
- To produce training instances, then simulate the operation of the parser by running the algorithm and relying on a new training oracle to give the correct transition operators for each successive configuration.

Creating an Oracle

- The parser begins with a default initial configuration {[ROOT], [w_1,w_2, … , w_n], []}
- Since we have a gold-standard reference parse for each training sentence, the dependency relations that are valid for a given sentence are known.
- ➔ We can use the reference parse to guide the selection of operators as the parser steps through a sequence of configurations.
- i.e., given a reference parse and a configuration, the training oracle proceeds as follows:
  - Choose LEFTARC if it produces a correct head-dependent relation given the reference parse and the current configuration,
  - Otherwise, choose RIGHTARC if (1) it produces a correct head-dependent relation given the reference parse and (2) all of the dependents of the word at the top of the stack have already been assigned,
  - Otherwise, choose SHIFT.

A Feature-based Classifier

- We subsequently introduce 2 classifiers for choosing transitions:
  - a classic feature-based algorithm and
  - a neural classifier using embedding features (next section).
- **Featured-based classifiers** generally use the features Word forms, lemmas, parts of speech, the head, and the dependency relation to the head.
- Other features may be relevant for some languages (e.g. morphosyntactic features like case marking on subjects or objects).
- The features are extracted from the training configurations (stack, buffer, and current set of relations).

Feature Generation using Templates

- A feature template is used. Feature templates allow us to automatically generate large numbers of specific features from a training set.
- For example, consider the following feature templates that are based on single positions in a configuration:

$$<s_1.w, op>, <s_2.w, op>, <s_1.t, op>, <s_2.t, op>,$$
$$<b_1.w, op>, <b_1.t, op>, <s_1.wt, op>$$

- Here features are denoted as location.property, where s = stack, b = word buffer, w = word forms, t = POS, and op = operator.
- E.g. feature for word form at top of stack is $s_1.w$; part of speech tag at the front of the buffer $b_1.t$, and the concatenated feature $s_1.wt$ represents the word form concatenated with the part of speech of the word at the top of the stack.

Feature Generation using Templates

- Example: Consider applying these templates to the following intermediate configuration derived from a training oracle (correct op being SHIFT):

Stack	Word buffer	Relations
[root, canceled, flights]	[to Houston]	(canceled → United) (flights → morning) (flights → the)

Feature-based Classification

- The application of the set of feature templates to this configuration would result in the following set of instantiated features:

  $<s_1.w = flights, op = shift>$

  $<s_2.w = canceled, op = shift>$

  $<s_1.t = NNS, op = shift>$

  $<s_2.t = VBD, op = shift>$

  $<b_1.w = to, op = shift>$

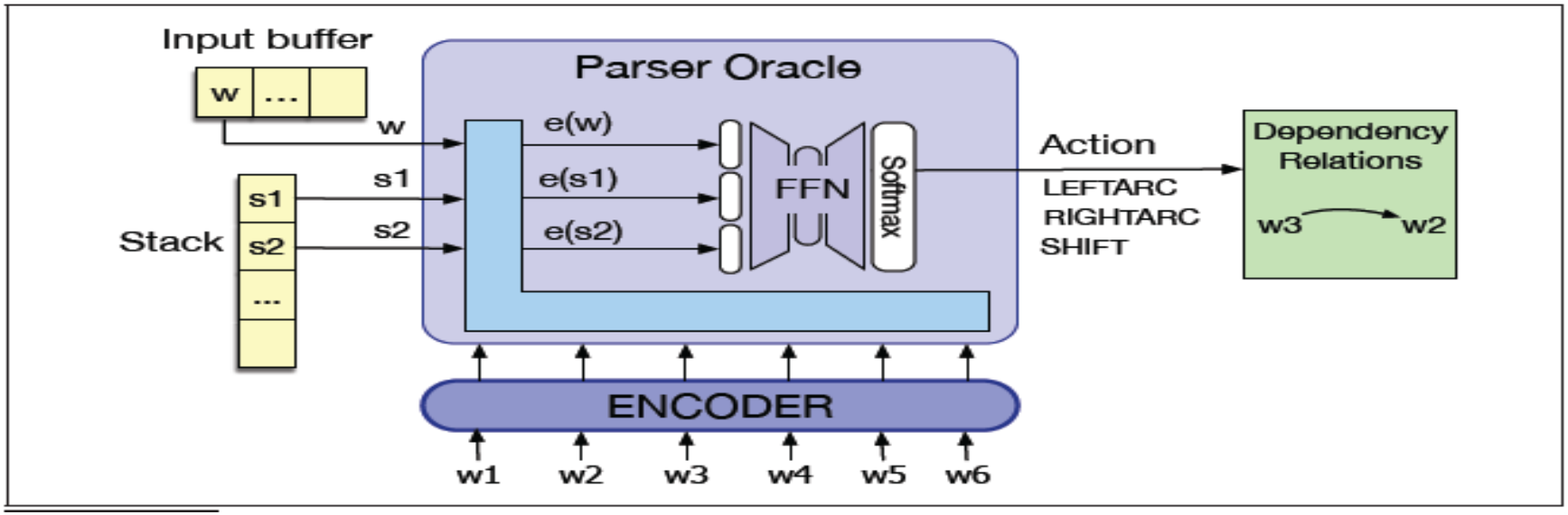
  $<b_1.t = TO, op = shift>$

  $<s_1.wt = flightsNNS, op = shift>$

- Given that the left and right arc transitions operate on the top two elements of the stack, features that combine properties from these positions are even more useful. E.g.: $<s_1.t \circ s_2.t = NNSVBD, op = shift>$
- Given the training data and features, any classifier, like multinomial logistic regression or support vector machines, can be used.

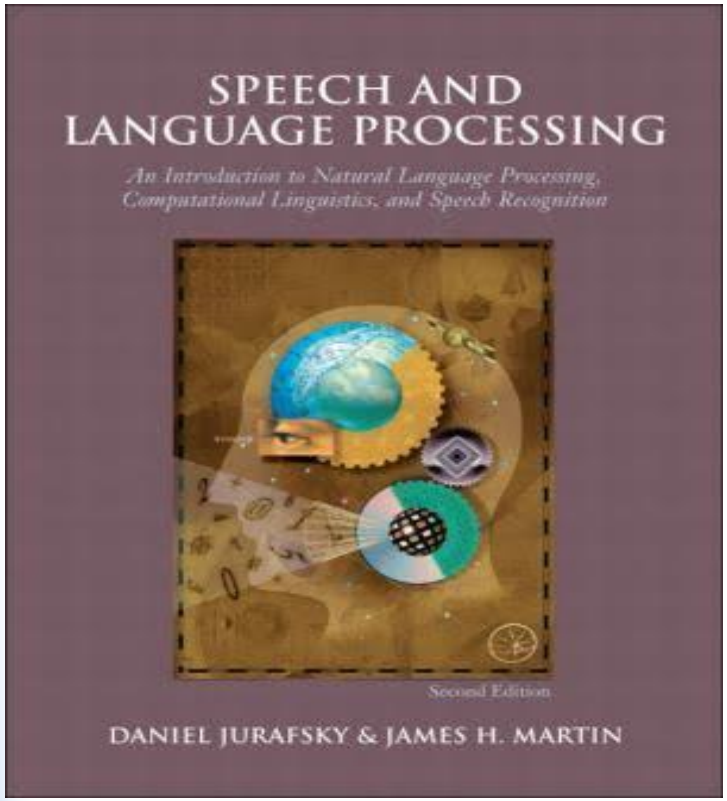
A neural classifier

- The oracle can also be implemented by a neural classifier.
- A standard architecture is simply to pass the sentence through an encoder, then take the presentation of the top 2 words on the stack and the first word of the buffer, concatenate them, and present to a feedforward network that predicts the transition to take.
- Learning can be done with cross-entropy loss.



Textbook

Jurafsky, D. and Martin, J.H. (2024) *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models* (3rd ed. draft), Prentice Hall.
https://web.stanford.edu/~jurafsky/slp3/ed3bookaug20_2024.pdf



- The draft version of the 3rd edition will be used; it has been updated to include more recent topics such as Vector Semantics and Embeddings, ANNs and Deep Learning for NLP, Transformers, etc.
- The **slides** are largely based on this textbook and its authors' slides with light customisations whenever seen fit.
- The slides with black background by Andrew Schwartz from Stony Brook University based on the same textbook (2021 version)